项目目录结构（Assets）

本文档描述 `Assets/` 目录下各文件夹的职责、更新机制和归属规则。所有文件夹按**更新机制**分为三个区域：AOT 区（编译进主包）、热更新区（运行时可更新）、编辑器区（仅开发时使用）。

一、目录总览

```
Assets/
├── Launcher/                    AOT     应用入口（主场景、启动配置、本地化）
├── Framework/                   AOT     基础框架（资源管理、网络、热更新引擎）
├── Game/
│   ├── Scripts/                 热更新  游戏代码（8 个热更新程序集 +
ThirdParty）
│   ├── HotResources/            热更新  可热更资源（Prefabs、Audio、RawAssets）
│   └── Res/                     打包     资源包构建源（构建时打入 AssetBundle）
├── Editor/                      编辑器  编辑器工具（构建、协议生成、SVN 等）
├── Resources/               AOT     Unity Resources（登录前多语言 JSON、IAP 配
置）
├── Plugins/                 AOT     原生插件（Android SDK、第三方 .dll）
├── StreamingAssets/          AOT     流式资源（AOT 补充元数据 DLL）
└── TextMesh Pro/            AOT     TextMeshPro 包资源
```

二、AOT 区（编译进主包）

AOT 区的内容编译进安装包，更新需重新提交应用商店。

1. Launcher

路径： `Assets/Launcher/`

应用启动入口，无 `.asmdef`，编译进默认 Assembly-CSharp。

文件	职责
`Main.unity`	Unity 主场景，挂载 Main 组件
`Main.cs`	入口 MonoBehaviour：检查热更新 → 加载程序集 → 启动 Gate
`Config.cs` + `Config.asset`	ScriptableObject：网关地址、版本号、环境切换
`Localization.cs`	登录前多语言（读取 Resources/ 下的 JSON）
`LanguageDetector.cs`	系统语言检测

2. Framework

路径： `Assets/Framework/` **程序集：** `Framework.asmdef`（AOT）

稳定基础设施，极少变更。

文件	职责
`Singleton.cs`	泛型单例基类
`AssetManager.cs`	资源加载管理
`Http.cs`	HTTP 请求
`Hot.cs`	热更新引擎
`Path.cs`	路径常量
`Loom.cs`	线程调度

3. Resources

路径： `Assets/Resources/`

Unity 特殊目录，通过 `Resources.Load()` 加载。仅存放登录前需要的静态数据。

内容：`Localization_{Language}.json`（23 种语言）、IAP 配置。

4. Plugins

路径： `Assets/Plugins/`

原生平台插件和预编译 DLL。子目录按平台划分（`Android/`、`libs/`）。

5. StreamingAssets

路径： `Assets/StreamingAssets/`

AOT 补充元数据 DLL（`Framework.dll.bytes` 等），用于 HybridCLR 运行时元数据注册。

6. TextMesh Pro

路径： `Assets/TextMesh Pro/`

Unity TextMeshPro 包的默认资源，含字体、Shader、示例。不修改。

三、热更新区（运行时可更新）

热更新区的内容可通过热更新机制单独下载更新，无需重新提交应用商店。

1. Game/Scripts

路径： `Assets/Game/Scripts/`

8 个热更新程序集 + 第三方库。详细分层架构见 `Documents/客户端架构文档.md`。

```
Scripts/
├── Utils/          工具层
├── Basic/          基础设施层（Event、Monitor、Flow）
├── Logic/          业务数据层（Data、Config、Models）
├── Network/        网络层（Net、Protocol、ProtocolConverter）
├── Domain/         领域层
├── UI/             表现层
├── Start/          启动层（Gate）
├── SDK/            平台 SDK
└── ThirdParty/     第三方库（LitJson、DOTween、SuperScrollView）
```

2. Game/HotResources

路径： `Assets/Game/HotResources/`

可热更新的游戏资源，构建为 AssetBundle 后随热更新包分发。

子目录	内容
`Audio/`	BGM、SFX 音频
`Prefabs/`	UI Prefab、特效 Prefab
`RawAssets/`	贴图、材质等原始资源

3. Game/Res

路径： `Assets/Game/Res/`

资源包构建源目录，构建时打入 AssetBundle。命名使用小写下划线格式。

四、编辑器区（仅开发时使用）

1. Editor

路径： `Assets/Editor/`

编辑器扩展工具，不编译进运行时包。

子目录	职责
`Build/`	构建脚本（Build.cs）
`Proto/`	Protocol 代码生成工具
`SVN/`	SVN 集成工具
`Tool/`	通用编辑器工具
`PlayerPrefs/`	PlayerPrefs 编辑器
`ExcelToLua/`	策划表导出

子目录	职责
`CopyPath/`	路径复制工具
`AutoAppend/`	自动追加工具

五、新增文件归属规则

新增文件或文件夹时，按以下规则判断归属：

问题	归属
是否为热更新游戏代码?	`Game/Scripts/{对应层}/`
是否为可热更新的资源（Prefab/音频/贴图）？	`Game/HotResources/`
是否为编辑器工具?	`Editor/`
是否为登录前需要的静态数据?	`Resources/`
是否为原生平台插件?	`Plugins/`
是否为启动流程代码?	`Launcher/`（极少修改）
是否为框架基础设施?	`Framework/`（极少修改）

禁止：

- 在 `Assets/` 根目录下新建顶层文件夹（除非有架构级别的理由）
- 将游戏代码放在 `Game/Scripts/` 以外的位置
- 将运行时资源放在 `Editor/` 下